

Relatório Técnico: Assistente Médico Virtual

Tech Challenge, Fase 3. Pós-graduação em Inteligência Artificial (FIAP)

Repositório: AI-TechChallenge-Step3

Sumário

1. Visão geral do sistema
2. Processo de fine-tuning
3. Descrição do assistente médico
4. Diagrama do fluxo LangChain / LangGraph
5. Avaliação do modelo e análise dos resultados
6. Referências e licenças

1. Visão geral do sistema

O sistema é um **assistente médico virtual de apoio à decisão clínica**. Ele atende **todas as especialidades** de um hospital fictício, o Hospital VEinstein.

O assistente não substitui o julgamento clínico. Ele organiza a informação do prontuário, recupera protocolos internos e redige respostas que precisam ser validadas por um profissional habilitado.

O escopo é hospitalar geral, e as três camadas de dados acompanham esse escopo. O prontuário eletrônico simulado, a base de protocolos indexada no RAG e o material de fine-tuning cobrem clínica médica, urgência, ortopedia, pneumologia, cardiologia, endocrinologia, gastroenterologia, urologia, neurologia, otorrinolaringologia, oftalmologia, dermatologia, infectologia, cirurgia geral, ginecologia e oncologia, além de temas transversais de segurança do paciente. O assistente é acionado da mesma forma para qualquer um desses casos.

A arquitetura combina três técnicas com papéis distintos:

Técnica	Papel	Por que é necessária
Fine-tuning (QLoRA)	Estilo, tom clínico, formato de resposta e vocabulário institucional	O modelo base responde com tom de enciclopédia geral, e o alvo é o registro clínico do hospital
RAG (LangChain + Chroma)	Conhecimento factual citável , vindo dos protocolos internos	Garante rastreabilidade: cada resposta indica de qual protocolo veio a informação
Tools sobre banco estruturado	Dado do paciente em tempo real : histórico, anamnese, exames	Esse dado muda a cada consulta e não pode estar congelado nos pesos do modelo

A orquestração é feita por um **grafo de estados em LangGraph**. Ele escolhe o caminho de execução conforme o tipo de interação e o resultado dos guardrails, e registra cada interação em um log de auditoria.

Números do sistema:

Item	Valor
Modelo base	Qwen/Qwen2.5-1.5B-Instruct (Apache 2.0)
Técnica de fine-tuning	QLoRA 4-bit (NF4) + LoRA r=16
Parâmetros treináveis	≈ 4,36 M (0,28% do total)
Dataset de fine-tuning	98 exemplos (84 treino, 14 validação)
Base de conhecimento (RAG)	32 protocolos, 122 chunks no Chroma
Prontuário simulado	32 pacientes, 30 quadros clínicos distintos, 38 exames, 32 anamneses
Grafo LangGraph	12 nós, 4 arestas condicionais
Endpoints da API	8
Testes automatizados	86

2. Processo de fine-tuning

2.1 Papel do fine-tuning na arquitetura

O que se ajusta no modelo é **comportamento**, não conhecimento. Um modelo de 1,5 bilhão de parâmetros responde perguntas médicas em português com tom de enciclopédia geral e formatação de chatbot. O comportamento desejado é outro: resposta curta, em registro clínico, que apresente qualquer conduta como sugestão sujeita à validação de um médico responsável e que cite o protocolo em que se apoia.

Esse tipo de ajuste envolve formato e tom, com pouco conteúdo novo. É justamente o caso em que LoRA e QLoRA funcionam bem com poucas dezenas de exemplos, e que o RAG não resolve. O RAG muda o que o modelo *sabe*, não como ele *escreve*.

2.2 Dataset

Duas pastas em `data/raw/`, `faqs/` e `laudos_modelo/`, alimentam o fine-tuning. Uma terceira pasta, `protocolos/`, tem um objetivo diferente: alimenta só o RAG e não entra no treino.

Fonte	Arquivos	Papel	Origem
<code>faqs/</code>	93	Pares instrução → resposta clínica	43 sintéticas internas (prefixo <code>hvn_</code>) e 50 derivadas de MedQuAD (NIH) e PubMedQA
<code>laudos_modelo/</code>	5	Formato e tom de documentos: laudo, parecer, encaminhamento, orientação pós-procedimento	Sintético, escrito para o hospital fictício
<code>protocolos/</code>	32	Fora do fine-tuning. É a base de conhecimento do RAG	Sintético

A separação entre o material de fine-tuning e o material de RAG é estrutural. Protocolo é *conhecimento a consultar e citar*. Treinar o modelo diretamente nos protocolos destruiria a rastreabilidade: ele passaria a "saber" o protocolo sem conseguir apontar a origem da informação, que é o requisito de explainability do desafio.

Composição das 93 FAQs por origem, identificável pelo prefixo do nome do arquivo:

Prefixo	Origem	Arquivos	Cobertura
hvn_	Sintético, interno ao hospital fictício	43	Todas as especialidades
cancergov_	MedQuAD / CancerGov	17	Oncologia mamária
pubmedqa_	PubMedQA	15	Oncologia mamária
seniorhealth_	MedQuAD / SeniorHealth	8	Oncologia mamária
gard_	MedQuAD / GARD	5	Oncologia mamária
ghr_	MedQuAD / Genetics Home Reference	3	Oncologia mamária
mplustopics_	MedQuAD / MedlinePlus Health Topics	2	Oncologia mamária

As 43 FAQs internas cobrem os quadros que estão de fato no prontuário simulado: hipertensão, diabetes, anemia, disfunção tireoidiana, dor crônica, dor abdominal aguda, cólica renal, trauma de extremidade, trombose venosa profunda, pneumonia, asma, tosse aguda, síndromes febris, infecção urinária, gastroenterite, refluxo, cefaleia, lombalgia, otite, sinusite, conjuntivite, dermatite e hérnia inguinal. Somam-se a esses os temas transversais de segurança: registro de alergias, interações medicamentosas, polifarmácia, sinais vitais de alerta, exames pendentes e uso racional de antimicrobianos. Cada FAQ referencia o protocolo interno correspondente, para que o modelo aprenda a citar a fonte institucional na resposta.

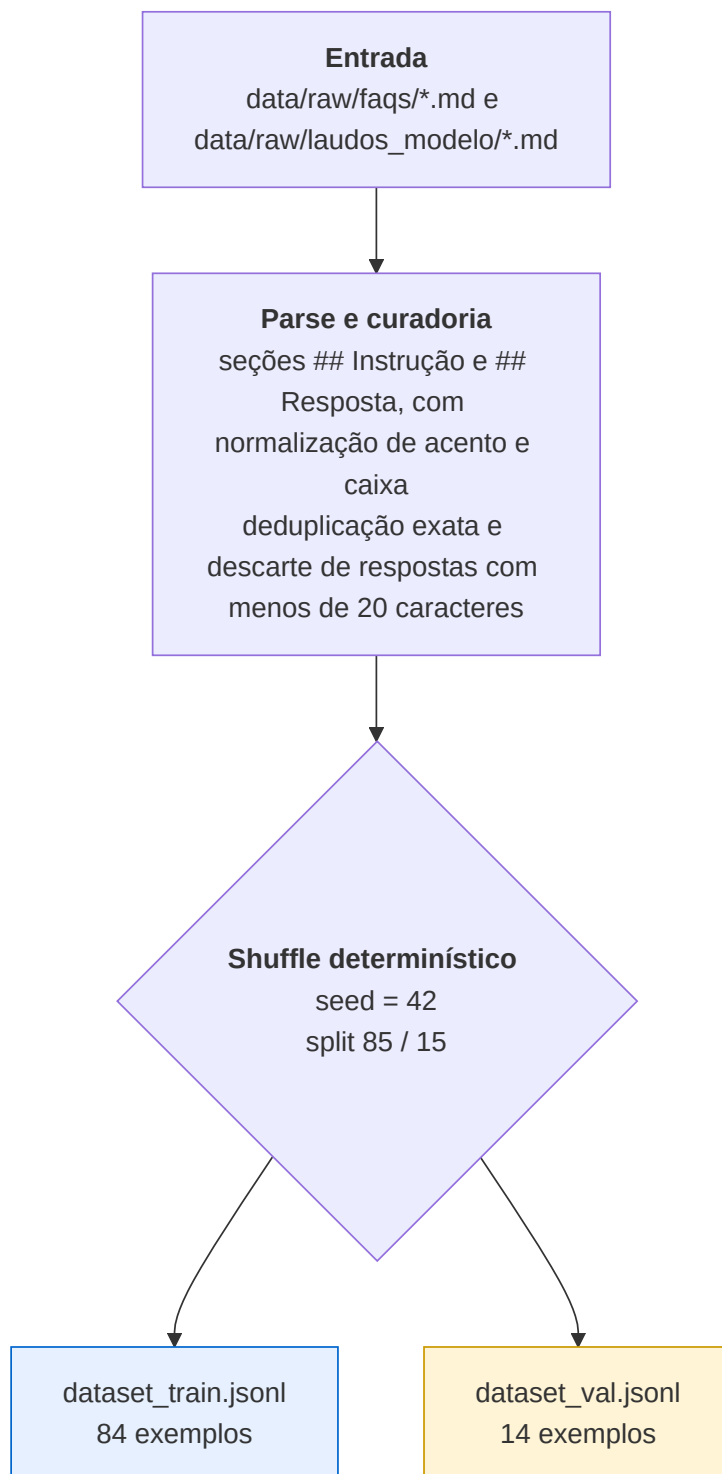
Nenhuma FAQ interna contém posologia específica. A conduta é sempre descrita em nível de orientação e classe terapêutica. Isso é coerente com o papel do assistente e com o guardrail de prescrição que verifica a saída: treinar o modelo com texto que o próprio sistema sinalizaria seria incoerente.

As 50 FAQs derivadas de bases públicas cobrem oncologia mamária. O PubMedQA tem formato original de pergunta de pesquisa, seguida de abstract e de uma conclusão sim, não ou talvez. Esse formato é incompatível com um par instrução → resposta clínica, então cada item foi reescrito como pergunta e resposta em prosa, preservando a conclusão do estudo. A tradução e o resumo para o português foram feitos com apoio de LLM, sem revisão por profissional de saúde. Esse conteúdo é material de estudo, não fonte validada para uso clínico.

Anonimização. Não há dado real de paciente em nenhuma etapa. MedQuAD e PubMedQA são bases públicas de conhecimento médico, sem informação identificável. O prontuário simulado é integralmente gerado, com nomes explicitamente fictícios ("Paciente Fictício A" até "Paciente Fictício AF"). A anonimização é estrutural: não existe dado sensível a anonimizar.

2.3 Preprocessing e curadoria

O script `finetuning/prepare_dataset.py` transforma os arquivos Markdown em JSONL no formato Alpaca (`instruction` , `input` , `output`):



Características da implementação:

- **Cabeçalhos tolerantes.** O parser aceita `## Instrução`, `## Pergunta` ou `## Contexto` para a entrada, e `## Resposta`, `## Laudo`, `## Receita`, `## Parecer` ou `## Encaminhamento` para a saída, normalizando acento e caixa. A fonte dos dados pode ser trocada sem alterar o código.
- **Split determinístico** (`RANDOM_STATE = 42`). O conjunto de validação é sempre o mesmo, o que torna as métricas da seção 5 comparáveis entre execuções.
- **Filtro de comprimento mínimo** (20 caracteres na resposta). Descarta respostas truncadas na conversão, que ensinariam o modelo a parar cedo.

Perfil do dataset resultante:

Métrica	Valor
Exemplos totais	98 (84 treino, 14 validação)
Palavras por instrução	média 13,7. Mínimo 6, máximo 22
Palavras por resposta	média 96,4. Mínimo 63, máximo 138
Volume total de resposta	≈ 9.400 palavras

2.4 Configuração do treino

Modelo base: `Qwen/Qwen2.5-1.5B-Instruct`. A escolha atende a três restrições ao mesmo tempo: licença Apache 2.0 sem gate de acesso, suporte multilíngue real ao português e capacidade de rodar em QLoRA 4-bit numa GPU comum.

Em **QLoRA**, o modelo base é quantizado em 4 bits (NF4, com dupla quantização) e congelado. Apenas os adaptadores de baixo posto (LoRA) recebem gradiente.

Hiperparâmetro	Valor	Justificativa
<code>LORA_R</code>	16	Posto suficiente para ajuste de estilo. Valores maiores aumentam o risco de overfit num dataset desta ordem
<code>LORA_ALPHA</code>	32	Razão $\alpha/r = 2$
<code>LORA_DROPOUT</code>	0.05	Regularização leve, apropriada ao dataset pequeno
<code>LORA_TARGET_MODULES</code>	<code>q_proj</code> , <code>k_proj</code> , <code>v_proj</code> , <code>o_proj</code>	Apenas as projeções de atenção. Incluir as camadas MLP aumentaria os parâmetros treináveis sem ganho claro para ajuste de tom
<code>LOAD_IN_4BIT</code>	<code>True</code> (NF4 + double quant)	Reduz a memória do modelo base o suficiente para caber na T4
<code>NUM_TRAIN_EPOCHS</code>	3	Com 84 exemplos, mais épocas memorizam o conjunto
Batch efetivo	8 (2 × 4 de acumulação)	Batch real de 2 cabe na VRAM da T4, e a acumulação recupera a estabilidade do gradiente
<code>LEARNING_RATE</code>	2e-4	Padrão para LoRA, bem acima do usado em fine-tuning completo
<code>MAX_SEQ_LENGTH</code>	1024	Acomoda o maior exemplo (system prompt, instrução e resposta)

Volume de treino: 84 exemplos divididos pelo batch efetivo de 8 dão 11 passos por época. Em 3 épocas, são **33 passos de otimização**.

Parâmetros treináveis: com $r=16$ sobre as quatro projeções de atenção, em 28 camadas de um modelo com `hidden_size=1536` e atenção agrupada (12 cabeças de consulta e 2 de chave/valor), o total é de aproximadamente **4,36 milhões de parâmetros**, ou **0,28%** dos cerca de 1,54 bilhão do modelo. O arquivo `adapter_model.safetensors` de 17,4 MB confirma a ordem de grandeza, já que 4,36 M multiplicados por 4 bytes dão aproximadamente 17,4 MB.

Formatação do prompt. O `SYSTEM_PROMPT` está definido em um único lugar (`finetuning/config.py`) e é usado tanto no treino (`train_qlora.py::build_prompt`) quanto na inferência (`rag/chain.py::ask` e `ask_overview`), sempre através do `apply_chat_template` do tokenizer. Treino e runtime usam exatamente o mesmo template, condição para que o adapter se comporte em produção como se comportou no treino.

O conteúdo do system prompt descreve os mesmos limites que o guardrail de runtime (`agent/guardrails.py`) verifica. O modelo é treinado para respeitar a regra que o sistema fiscaliza. A coerência vai até o dataset: nenhum dos 98 exemplos de treino dispara o filtro de prescrição.

2.5 Execução e artefato

O treino roda localmente, com o script `finetuning/train_qlora.py`. A stack é `transformers`, `peft 0.20`, `trl 1.12` (SFTTrainer), `bitsandbytes` e `accelerate`.

Sem GPU, o script detecta a ausência de CUDA e carrega o modelo sem quantização em `bfloat16`. Isso é LoRA e não QLoRA, já que o `bitsandbytes` 4-bit exige CUDA. Nesse caminho, o *gradient checkpointing* é habilitado para compensar a memória. Com GPU, o dtype de computação é resolvido automaticamente: `bfloat16` em Ampere ou superior, e `float16` em GPUs mais antigas, que não suportam `bf16` nativamente.

Artefato final: a pasta `finetuning/adapters/cancer-assistant-lora/`, com o adapter LoRA de 17,4 MB, o tokenizer e o chat template. O modelo base não é redistribuído. Ele é baixado do Hugging Face na primeira execução.

3. Descrição do assistente médico

3.1 Modos de operação

O assistente vive dentro do painel do médico (Angular 20) e opera em **dois modos**, com caminhos distintos no grafo.

Visão geral automática. Ao abrir a ficha de um paciente, o front-end chama `POST /patients/{id}/overview`. O assistente lê o histórico, a anamnese completa e os exames, e devolve duas listas estruturadas: **pontos relevantes**, que resumem o caso, e **pontos de atenção**, com riscos, exames pendentes, alergias, interações medicamentosas e divergências entre histórico e conduta. O médico vê isso como um card ao lado da ficha, sem precisar perguntar nada.

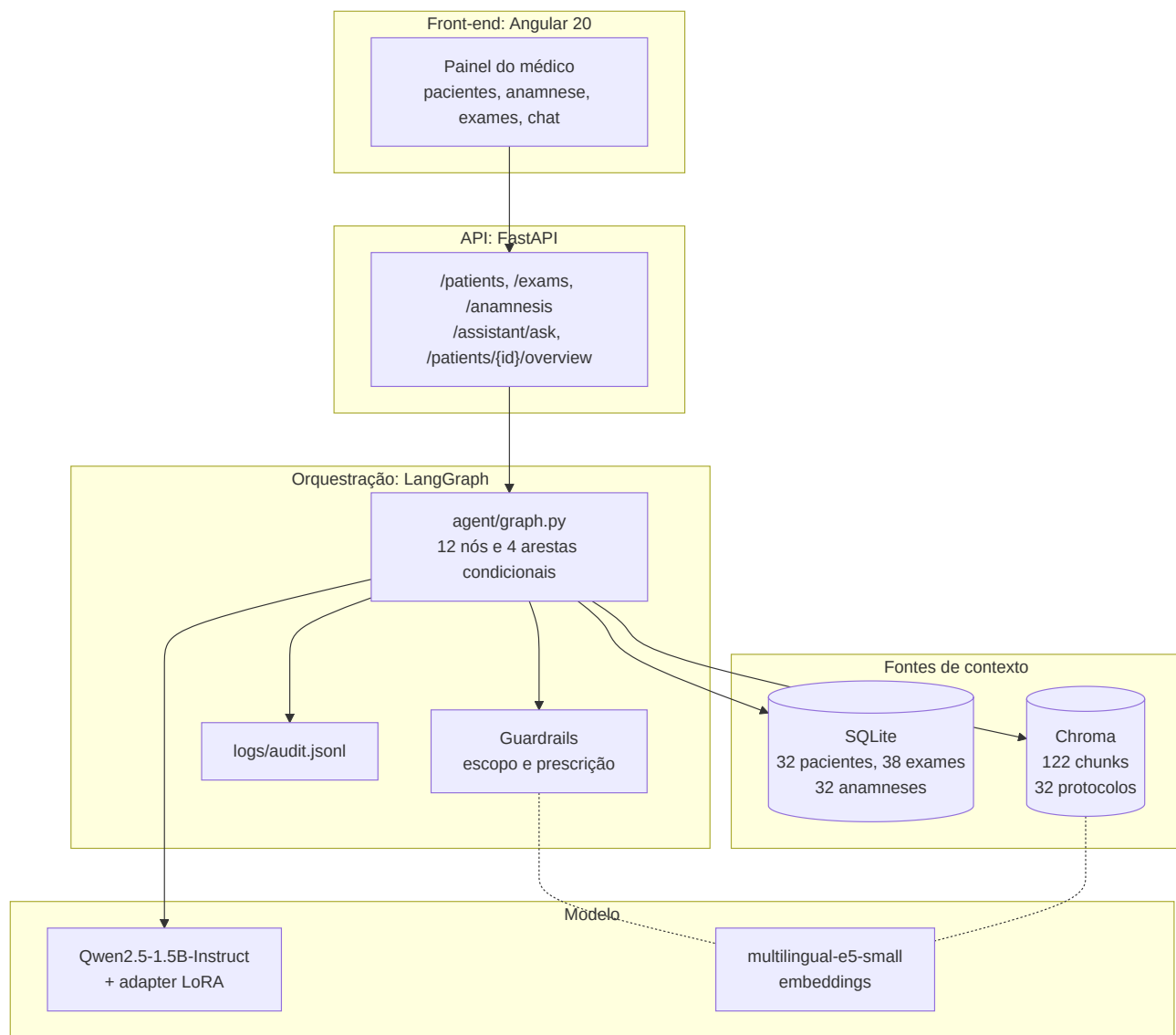
Chat. O médico faz uma pergunta livre sobre o paciente em consulta (`POST /assistant/ask`). A resposta é ancorada nos protocolos internos recuperados via RAG e traz a lista de protocolos consultados.

Nos dois modos, exames pendentes geram um alerta emitido antes da geração, que acompanha a resposta.

Os dois modos são caminhos separados porque a tarefa é diferente. O chat responde a uma pergunta e precisa de retrieval. A visão geral sintetiza um caso e usa um prompt de saída estruturada, com os marcadores fixos `RELEVANTE:` e `ATENCAO:` que o backend converte em duas listas. Os nós comuns,

como checagem de exames, guardrails, validação humana e auditoria, são compartilhados pelos dois caminhos.

3.2 Componentes



Componente	Tecnologia	Papel
Orquestração	LangGraph	Grafo de estados. Garante que todo caminho passe por guardrail e auditoria
Geração	<code>transformers</code> e <code>peft</code> , via <code>HuggingFacePipeline</code>	Modelo base com adapter LoRA, servido em processo
Recuperação	LangChain e Chroma	122 chunks (500 caracteres, overlap de 50) dos 32 protocolos, com metadado <code>fonte</code> . O corte de relevância fica em <code>rag/relevance.py</code>
Embeddings	<code>intfloat/multilingual-e5-small</code>	Usado no RAG e no guardrail de escopo
Prontuário	SQLite	Tabelas <code>pacientes</code> , <code>exams</code> e <code>anamnese</code> , com seed idempotente

Componente	Tecnologia	Papel
API	FastAPI e Pydantic	Grafo carregado uma vez no <code>lifespan</code> , não a cada requisição
Front-end	Angular 20 (standalone e Signals) com Material	Painel do médico

A família `Intfloat/multilingual-e5-*` exige os prefixos `passage:` para documentos e `query:` para buscas. Isso está encapsulado em uma subclasse (`rag/embeddings.py::E5Embeddings`) usada tanto na ingestão quanto na busca. Se apenas um dos lados aplicasse o prefixo, a qualidade de recuperação cairia sem gerar erro.

3.3 Prontuário eletrônico simulado

Sem acesso a um sistema hospitalar real, o prontuário é um SQLite consultado como tool do agente, e não embutido no prompt. São **32 pacientes cobrindo 30 quadros clínicos distintos**, cada um com anamnese completa e exames coerentes com o diagnóstico:

Área	Quadros representados
Clínica médica	Hipertensão descompensada, diabetes tipo 2 descompensado, hipotireoidismo, anemia ferropriva, fibromialgia
Urgência	Apendicite aguda, cólica renal por litíase ureteral, fratura de rádio distal, suspeita de trombose venosa profunda
Pneumologia	Pneumonia adquirida na comunidade, bronquite aguda, crise asmática
Infectologia	Síndrome gripal, COVID-19, dengue, infecção do trato urinário, sinusite bacteriana
Gastroenterologia	Gastroenterite aguda, doença do refluxo gastroesofágico
Cirurgia geral	Hérnia inguinal não complicada
Neurologia	Enxaqueca crônica, lombociatalgia com suspeita de hérnia de disco
Otorrinolaringologia e oftalmologia	Otite média aguda, conjuntivite bacteriana
Dermatologia	Dermatite de contato alérgica
Ginecologia e oncologia	Nódulo mamário em investigação, cisto mamário benigno, carcinoma ductal invasivo em tratamento
Rotina	Paciente hígido, rastreamento ginecológico e mamário

Schema:

- `pacientes`: id, nome fictício, histórico, sexo e data de nascimento. A idade não é coluna. Ela é calculada a partir da data de nascimento a cada consulta, para não desatualizar.
- `exames`: id, paciente, tipo, status (`pendente` ou `concluido`), datas de solicitação e realização, resultado, médico solicitante e observações.
- `anamnese`: um registro por paciente, com queixa principal, história da doença atual, história patológica pregressa, história familiar, história ginecológica e obstétrica, medicamentos em uso,

alergias, cirurgias prévias, hábitos, sinais vitais, peso e altura, exame físico, hipótese diagnóstica e conduta.

O seed é idempotente por id (`INSERT OR IGNORE`). Reiniciar a API preenche apenas o que falta, sem duplicar nem sobrescrever, e migra bancos criados por versões anteriores do schema.

Contexto enviado ao modelo. A função `montar_contexto_clinico` monta o bloco que vai ao prompt, e ele **abre com idade e sexo**. São os dois dados que mais condicionam conduta: para um mesmo diagnóstico, a apresentação, a investigação e o procedimento indicado mudam completamente entre uma criança e um idoso. Um modelo que não recebe a idade preenche a lacuna com a faixa etária mais frequente na literatura do diagnóstico, e produz conduta correta para o diagnóstico e errada para o paciente. A [seção 5.6](#) detalha esse caso. Depois da identificação vêm o histórico resumido, os campos da anamnese e os sinais vitais em linha condensada.

3.4 Base de protocolos internos

A base de conhecimento consultada pelo RAG são **32 protocolos** clínicos sintéticos do hospital fictício, indexados por `rag/ingest.py`. Eles cobrem as mesmas áreas presentes no prontuário, mais temas transversais de segurança do paciente:

Código	Área	Conteúdo
MAMA-01 , MAMA-02 , ONCO-01 , ONCO-03	Oncologia mamária	Rastreamento, conduta por BI-RADS, encaminhamento, segurança em quimioterapia
CLI-01 a CLI-05	Clínica médica	Hipertensão, diabetes tipo 2, anemia, disfunção tireoidiana, dor crônica generalizada
URG-01 a URG-04	Urgência	Dor abdominal aguda, cólica renal, trauma de extremidade, suspeita de trombose venosa profunda
PNE-01 a PNE-03	Pneumologia	Pneumonia adquirida na comunidade, asma, tosse aguda
INF-01 a INF-03	Infectologia	Triagem de síndromes febris, infecção urinária, uso racional de antimicrobianos
GAS-01 , GAS-02	Gastroenterologia	Gastroenterite e hidratação, doença do refluxo
NEU-01 , NEU-02	Neurologia	Cefaleia, lombalgia e lombociatalgia
OTO-01 , OFT-01 , DER-01	Otorrino, oftalmologia e dermatologia	Otite e sinusite, olho vermelho, dermatite de contato
CIR-01	Cirurgia geral	Hérnia inguinal e avaliação cirúrgica eletiva
SEG-01 a SEG-04	Segurança do paciente	Alergias e interações, sinais vitais de alerta, exames pendentes, contraste em exames de imagem
ADM-01	Rotina	Consulta de check-up e rastreamento

Cada protocolo é estruturado em torno dos pontos que mudam a conduta: critérios diagnósticos, sinais de alarme que exigem escalonamento e a ressalva explícita de que a decisão final é do médico responsável. Vários protocolos referenciam outros pelo código, o que dá ao retriever mais de um caminho até a informação pertinente.

Nenhum protocolo contém posologia específica, porque o assistente é de apoio à decisão e não prescreve.

Reindexação. O `rag/ingest.py` reconstrói o índice do zero a cada execução, pela função `reset_vectorstore`. Isso é necessário porque o `Chroma.from_documents` sobre um `persist_directory` existente **acrescenta** documentos à coleção em vez de substituí-los. Sem o reset, cada execução da ingestão duplica todos os chunks, e o efeito na recuperação é silencioso e grave. A [seção 5.4](#) mostra a medição.

3.5 API

Método	Rota	Descrição
GET	/	Raiz. Confirma que a API está no ar e aponta para <code>/docs</code>
GET	/health	Status da API e se o grafo foi carregado
GET	/patients	Lista pacientes, com sexo, data de nascimento e idade. O parâmetro <code>?nome=</code> filtra por parte do nome
GET	/patients/{id}/exams	Exames do paciente com status e datas
GET	/exams/{id}	Detalhe completo de um exame
GET	/patients/{id}/anamnesis	Ficha de anamnese completa
POST	/assistant/ask	Pergunta livre do médico. Devolve a resposta, as fontes citadas, <code>termos_etarios_incoerentes</code> e <code>citacoes_invalidas</code>
POST	/patients/{id}/overview	Visão geral automática. Devolve <code>pontos_relevantes</code> e <code>pontos_atencao</code>

Convenção de erros: `503` quando falta um artefato de que o assistente depende, como o índice do RAG ou o adapter treinado. `404` para paciente ou exame inexistente. `500` para o restante.

3.6 Segurança, guardrails e explainability

Oito camadas cobrem os requisitos de limite de atuação, logging e explainability.

1. System prompt compartilhado. Um único texto, em `finetuning/config.py`, usado no treino e no runtime. Ele impõe três regras: usar o contexto fornecido, dizer quando o contexto for insuficiente e **nunca prescrever diretamente**. A conduta sempre aparece como recomendação, a ser validada por um médico responsável.

2. Guardrail de escopo (`agent/scope_guard.py`). É uma camada anterior à geração, aplicada ao caminho do chat. Ela classifica a pergunta por similaridade de embeddings **antes** de montar o contexto do paciente. A classificação é **relativa**: a pergunta é comparada contra um conjunto de âncoras do domínio clínico e contra um conjunto de âncoras fora do domínio, como viagem, veículos, esporte, clima, culinária e finanças. A pergunta só é aceita quando a similaridade com o lado clínico supera a do lado fora de escopo por uma margem mínima, definida em `SCOPE_MARGIN`.

A comparação é relativa, não contra um limiar fixo. O `multilingual-e5-small` já dá similaridade alta entre duas frases curtas em português, só pela forma de pergunta e pelo tom formal. Comparar os dois

lados cancela esse piso comum. Sobra só o sinal de tópico, que é o que importa. Perguntas fora do escopo vão direto para uma resposta fixa, escrita em código. Essa resposta **nunca chega a ver o prontuário**. Assim o modelo não mistura dado clínico numa recusa.

3. Guardrail de prescrição (`agent/guardrails.py`). É um filtro de expressões regulares aplicado a **toda** resposta, em todos os caminhos. Ele procura linguagem de prescrição direta, como `prescrevo` , `receito` , `tome ... mg` e `administre ... dose` . A resposta nunca é descartada em silêncio: se algum padrão dispara, ela é marcada com `requer_validacao_humana=True` e recebe um aviso anexado, de modo que o médico vê o rascunho sinalizado.

A opção por regex, e não por classificador de ML, é deliberada. Em contexto clínico, um filtro simples e auditável é mais defensável que uma caixa-preta sem explicação. Ele é conservador por construção: prefere o falso-positivo ao falso-negativo.

4. Checagem de coerência etária (`agent/demographic_guard.py`). Aplicada a toda resposta. Procura termos de faixa etária, como `recém-nascido` , `lactente` , `criança` , `pediátrico` , `adolescente` , `idoso` e `geriátrico` . Sinaliza a resposta quando nenhum desses termos combina com a idade do paciente.

A regra não é "mencionou uma faixa que não é a do paciente". A regra é "mencionou faixas etárias, e nenhuma delas contém a idade do paciente". A distinção importa. Veja este exemplo: *"diferente do que ocorre em crianças, no idoso a conduta é..."*. Essa frase cita uma faixa incompatível (crianças), mas cita também a faixa certa (idoso), o que mostra que a resposta está orientada pela idade certa. O erro que o guardrail quer pegar é outro: a resposta inteira ancorada na faixa errada, sem nenhum termo compatível. Sinalizar os dois casos geraria alarme numa resposta correta, e um guardrail que dispara em resposta boa deixa de ser lido.

Como o guardrail de prescrição, esta camada **não bloqueia nem reescreve**. Ela marca `requer_validacao_humana=True` e anexa a ressalva. Os termos que dispararam vão para o log de auditoria e para a resposta da API, de forma que o revisor sabe exatamente o que procurar no texto.

5. Verificação de citações (`agent/citation_guard.py`). Aplicada ao caminho do chat. Confere se os protocolos citados na resposta são os mesmos que foram de fato recuperados para aquela pergunta. Ela extrai da resposta as referências no formato `protocolo <código>` e compara com os códigos derivados dos nomes de arquivo das fontes. Por exemplo, `protocolo_cir01_hernia_inguinal` vira `CIR-01` .

A citação é o que dá autoridade institucional a uma afirmação. Por isso, uma citação errada é pior do que nenhuma citação: se a resposta atribui a um protocolo algo que ele não diz, o médico só percebe abrindo o protocolo. A checagem pega dois casos, o código que não existe e o código real que não foi consultado para aquela pergunta. Nos dois casos, o modelo cita algo que não poderia ter lido.

O que ela não cobre é o protocolo citado corretamente, mas cujo conteúdo a resposta descreve errado. Pegar esse erro exigiria comparar a afirmação com o texto do protocolo, uma tarefa semântica, fora do alcance de uma checagem léxica como essa. Não citar protocolo nenhum nunca é sinalizado, porque às vezes não há protocolo aplicável.

6. Filtro de ancoragem da visão geral (`agent/overview_filter.py`). É a última camada antes de os pontos gerados chegarem ao médico, e só se aplica ao caminho da visão geral. O parser cuida da forma da resposta. Este filtro cuida do conteúdo: um ponto só aparece se puder ser rastreado ao

prontuário daquele paciente, ou seja, se compartilhar ao menos um termo com a ficha clínica. A comparação usa um radical de cinco caracteres, sem acento e sem maiúsculas, para tolerar variações de palavra sem depender de um lematizador.

O filtro também garante em código o teto de 5 itens por bloco que o prompt pede. Ele junta famílias de palavras parecidas: três ou mais itens com o mesmo radical viram um só. E marca a geração como **degenerada** quando ela tem itens demais, ou poucos itens ancorados no prontuário. Numa geração degenerada, o bloco inteiro é descartado, não só os itens sem âncora. O motivo: os poucos itens ancorados podem ter combinado por coincidência de palavra, e o médico não tem como saber quais vieram de qual regime de geração.

Essa regra tem um custo assumido: ela também descarta inferência clínica legítima, quando usa vocabulário ausente da ficha. Mas num apoio à decisão clínica, mostrar um achado fabricado custa mais do que esconder um achado correto, que o médico consegue obter lendo a própria ficha e o protocolo. O número de itens descartados e o sinalizador de degeneração vão para o log de auditoria e para a resposta da API. Assim esse custo fica mensurável, em vez de invisível.

7. Nó de validação humana. Respostas sinalizadas passam por um nó dedicado, que marca `status="aguardando_validacao_humana"`. O fluxo não é interrompido: a decisão sobre um rascunho sinalizado é do médico, não do sistema.

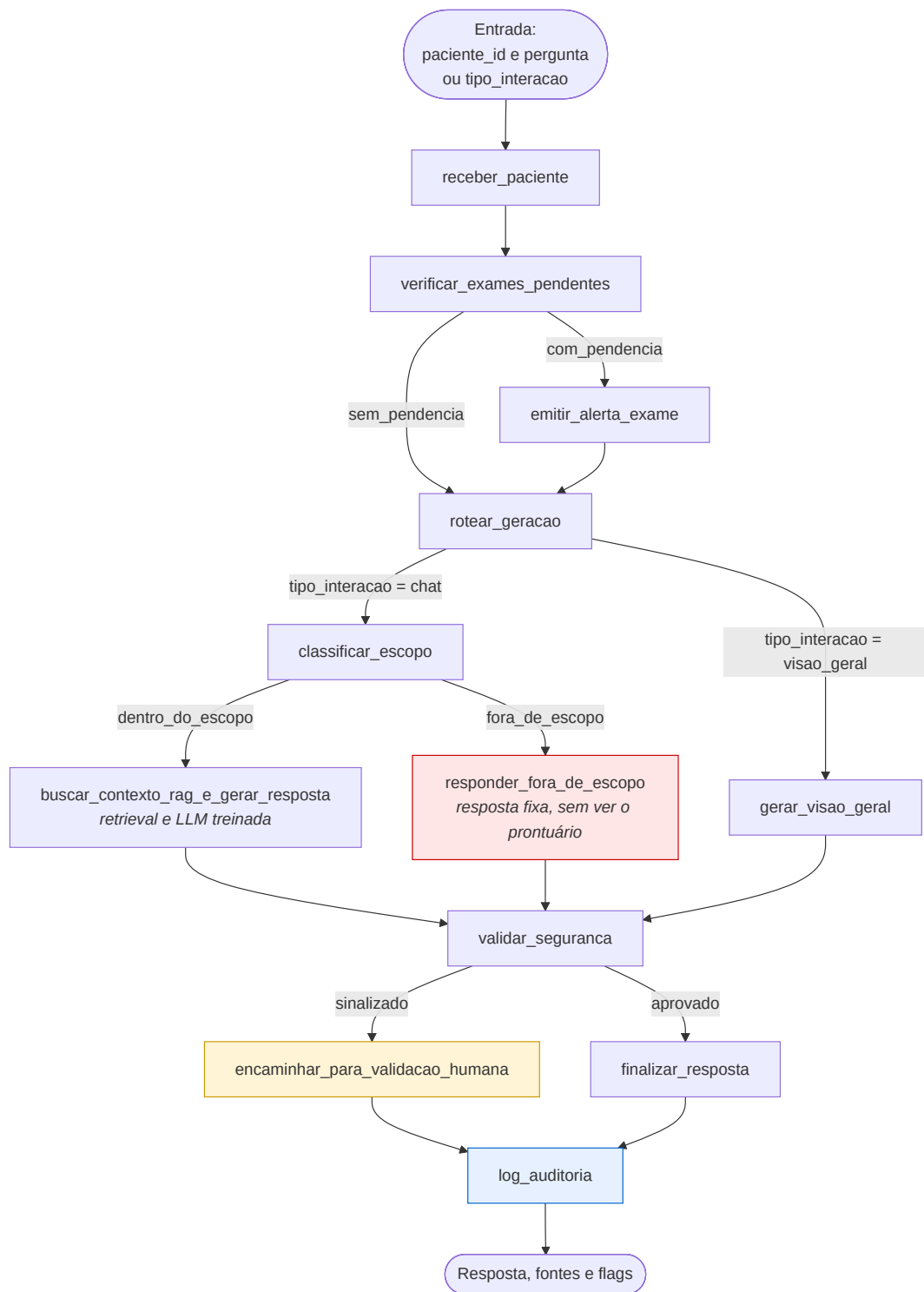
8. Explainability em duas frentes. Toda resposta de chat cita os protocolos recuperados. O metadado `fonte` é preservado desde a leitura do arquivo, antes do chunking, e volta no campo `fontes` da API.

Além disso, o log de auditoria (`logs/audit.jsonl`) grava **24 campos por interação**, organizados em quatro grupos: dados do caso (timestamp, tipo de interação, paciente e idade, pergunta e decisão), evidência de recuperação (as três similaridades do guardrail de escopo, exames pendentes, alerta, fontes citadas e as similaridades dos chunks aceitos), conteúdo gerado (resposta bruta, resposta final, pontos estruturados da visão geral e quantos foram descartados) e sinalizadores de guardrail (padrões de prescrição, termos etários incoerentes, citações inválidas, geração degenerada e status).

Registrar a resposta **bruta** ao lado da final é o que torna cada camada auditável depois do fato. Dá para reconstruir o que o modelo produziu, o que cada guardrail mudou e por quê, e foi assim que os casos analisados na seção 5 foram diagnosticados. O log grava uma linha JSON por interação, sempre no final do arquivo, e uma falha de escrita nunca derruba uma resposta já gerada.

4. Diagrama do fluxo LangChain / LangGraph

4.1 Grafo de decisão (LangGraph)



Arestas condicionais:

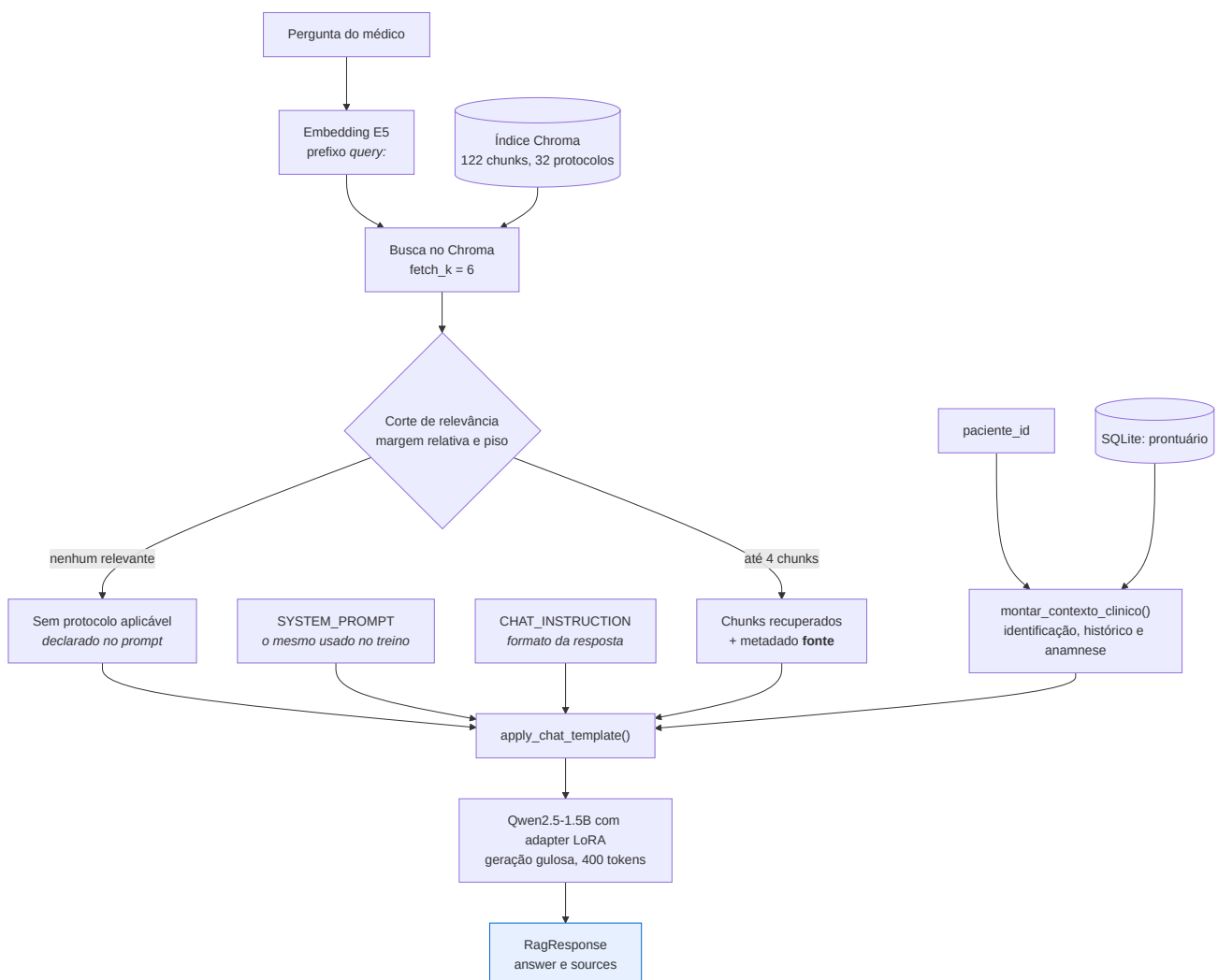
Aresta	Decisão baseada em	Caminhos
verificar_exames_pendentes	Consulta ao SQLite	com_pendencia emite alerta. sem_pendencia segue
rotear_geracao	Campo tipo_interacao do estado	chat vai para a classificação de escopo. visao_geral vai para a geração direta

Aresta	Decisão baseada em	Caminhos
<code>classificar_escopo</code>	Similaridade relativa de embeddings	<code>dentro_do_escopo</code> vai para RAG e geração. <code>fora_de_escopo</code> vai para a resposta fixa
<code>validar_seguranca</code>	Resultado do filtro de prescrição	<code>sinalizado</code> vai para validação humana. <code>aprovado</code> finaliza

Três propriedades que o desenho garante:

1. O **`log_auditoria`** é terminal e sempre executado. Não existe caminho de saída que não passe por ele, nem o de recusa por escopo, nem o de resposta sinalizada.
2. O **`validar_seguranca`** é ponto de convergência obrigatório. Os três nós de geração (RAG, visão geral e recusa) convergem nele, e nenhuma resposta escapa do guardrail.
3. O **`rotear_geracao`** é um nó de passagem, sem efeito no estado. Ele existe para dar um ponto único de saída aos dois caminhos anteriores, com e sem exame pendente, o que evita duplicar a aresta condicional de tipo de interação.

4.2 Chain de recuperação e geração (LangChain)

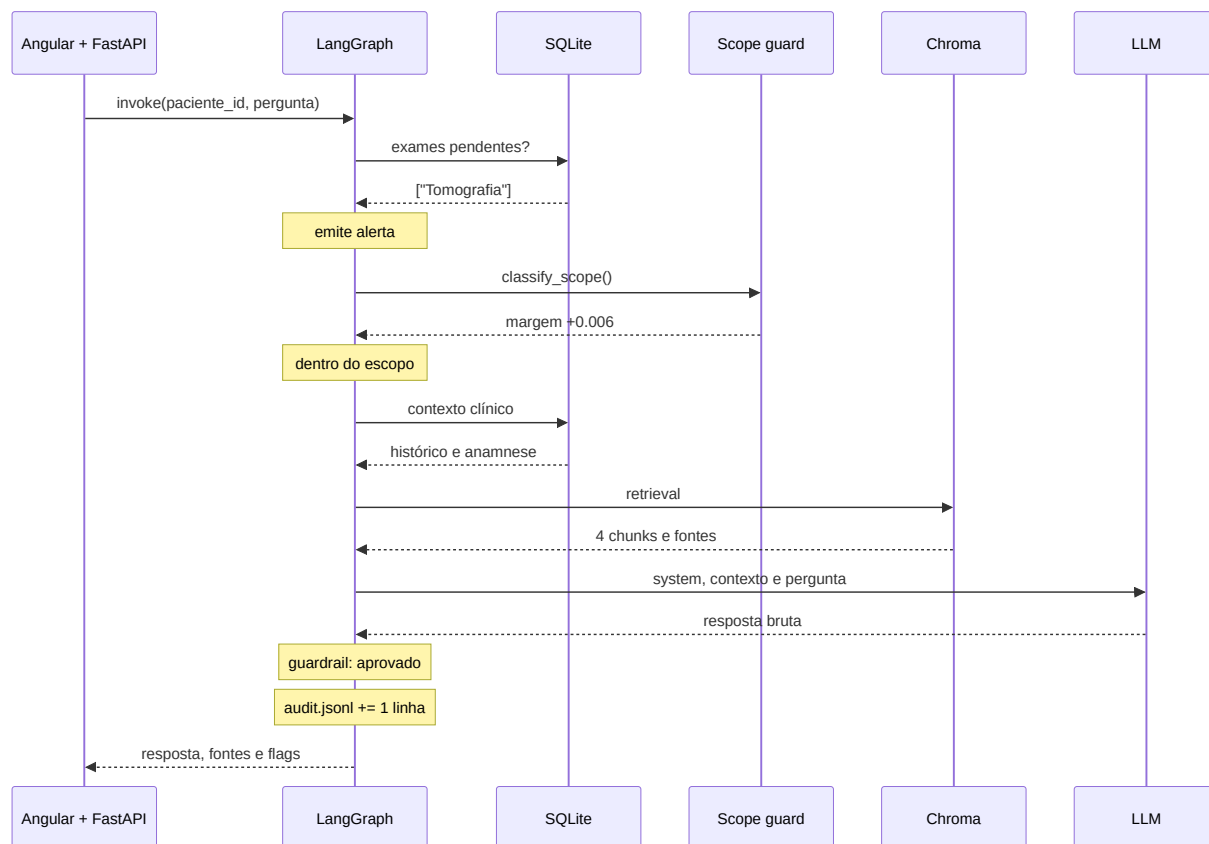


Quatro características da implementação:

- **A chain não é uma única Runnable LCEL encadeada.** Ela é montada como um dicionário de componentes (`vectorstore` , `llm_chat` , `llm_overview` , `tokenizer`), porque a função `ask()` precisa da resposta gerada e dos metadados de fonte dos chunks. Essa informação se perde numa chain LCEL que devolve apenas a string final. A explainability exigida pelo desafio determina a estrutura da chain.
- **Dois wrappers de pipeline sobre o mesmo modelo.** O `llm_chat` (400 tokens) e o `llm_overview` (450 tokens) compartilham o mesmo `model` e o mesmo `tokenizer` já carregados. Não há peso duplicado em memória, apenas duas configurações de geração. Ambos usam **geração gulosa** (`do_sample=False`), `repetition_penalty=1.15` e `no_repeat_ngram_size=3` . Responder uma pergunta clínica a partir de um prontuário e de um protocolo não é tarefa criativa. A variação entre execuções não produz resposta melhor, atrapalha a auditoria, já que a mesma pergunta sobre o mesmo paciente deveria dar a mesma resposta, e num modelo pequeno é por onde entram palavras coladas e malformadas.
- **Corte de relevância na recuperação** (`rag/relevance.py`). A busca traz 6 candidatos, e o corte decide quantos entram no prompt. São aceitos os que estiverem a até 0,05 de similaridade do melhor, com limite de 4. O corte é **relativo** pela mesma razão do guardrail de escopo: o E5 tem piso alto de similaridade entre textos curtos em português, e um limiar fixo ou aprova tudo ou corta tudo, conforme o comprimento da pergunta. Como o corte relativo sozinho aprovaria em bloco um conjunto de candidatos igualmente ruins, há também um piso absoluto baixo. Se nem o melhor candidato o supera, a lista volta vazia e o prompt declara que nenhum protocolo interno trata do assunto.
- **Formato de resposta pedido ao chat** (`CHAT_INSTRUCTION`). Conduta sugerida em uma frase direta, justificativa em até três frases, citação apenas dos protocolos que se aplicam, proibição de contradizer a ficha do paciente e de sustentar duas condutas incompatíveis, e sinais de alerta ao final.

A visão geral usa uma variante desse fluxo, **sem retrieval**, porque o insumo é o caso do paciente e não os protocolos do hospital.

4.3 Sequência ponta a ponta (chat)



5. Avaliação do modelo e análise dos resultados

A avaliação tem duas frentes. A primeira é a **comparação quantitativa** entre o modelo base e o modelo treinado. A segunda é a **medição do comportamento do sistema em operação**, a partir das 68 interações registradas no log de auditoria.

As medições de operação das seções 5.3 a 5.7 foram obtidas com a base de protocolos e o adapter anteriores à ampliação para as demais especialidades. Elas continuam válidas como caracterização das camadas que não mudaram, como o guardrail de escopo, o parsing e a auditoria, e como diagnóstico da configuração de recuperação. A [seção 5.9](#) registra o que precisa ser medido de novo depois do re-treino e da reindexação.

5.1 Metodologia da comparação base vs. fine-tuned

O `finetuning/evaluate.py` gera, para o mesmo conjunto de perguntas, as respostas do **modelo base sem adapter** e do **modelo base com adapter LoRA**, e calcula métricas comparáveis.

Conjunto de teste: as 14 perguntas de `dataset_val.jsonl`. Esse é o split de validação, nunca usado para atualizar pesos, já que o `SFTTrainer` o consome apenas como `eval_dataset` para a loss. Por ser sorteado do conjunto completo com semente fixa, ele cobre perguntas de oncologia mamária, das demais especialidades e dos temas transversais de segurança, além dos pedidos de geração de documento.

Métricas. Nenhuma delas mede correção clínica, o que exigiria um médico avaliando as respostas e está fora do escopo deste trabalho. Elas medem se o fine-tuning aproximou o **comportamento do**

modelo do comportamento desejado:

Métrica	O que mede	Direção
<code>similaridade_referencia</code>	Cosseno entre o embedding da resposta gerada e o da resposta de referência, com o mesmo E5 do RAG. É um proxy de "tratou do mesmo assunto", não de "acertou"	maior
<code>taxa_guardrail</code>	Fração de respostas que o filtro de prescrição sinalizaria. Reusa o mesmo <code>agent/guardrails.py</code> do runtime	menor
<code>taxa_ressalva</code>	Fração de respostas que apresentam a informação como sujeita a validação profissional	maior
<code>distinct_3</code>	Proporção de trigramas distintos. Detecta degeneração por repetição	maior
<code>comprimento_palavras</code>	Média de palavras por resposta. Indica mudança no regime de verbosidade	neutra

Controles metodológicos:

- **Geração gulosa** (`do_sample=False`), para que a comparação seja reproduzível entre execuções.
- **Sem RAG**, para isolar o efeito do fine-tuning. As respostas da API passam ainda por retrieval e pelo grafo.
- **Mesmo prompt e mesmo chat template** dos dois lados, idênticos aos do treino. Um template divergente mediria a divergência de template, e não o efeito do fine-tuning.
- **Modelos carregados em sequência**, e não ao mesmo tempo. Em CPU, manter os dois na RAM dobraria o consumo sem necessidade.
- O `multilingual-e5-small` tem piso alto de similaridade entre textos em português, então o valor absoluto importa menos que a **diferença** entre base e treinado no mesmo item.

```
python finetuning/evaluate.py
# gera data/processed/evaluation/evaluation.json (respostas completas e métricas por item)
# gera data/processed/evaluation/evaluation.md (tabela comparativa)
```

5.2 Resultados da comparação

Métrica	Base	Fine-tuned	Δ	Melhor
Similaridade com a referência (cosseno E5)	0,8993	0,9013	+0,002	maior
Taxa de respostas sinalizadas pelo guardrail	0,0	0,0	+0,0	menor
Taxa de respostas com ressalva de validação humana	0,2857	0,2143	-0,0714	maior
Trigramas distintos (1.0 = sem repetição)	0,9468	0,9389	-0,0079	maior
Comprimento médio (palavras)	189,8	107,1	-82,7	neutra

Hipóteses declaradas antes da execução, para que o resultado seja interpretável e não retroajustado:

1. A **taxa de ressalva** deve subir no modelo treinado. É o comportamento mais diretamente ensinado pelo system prompt, repetido em 84 exemplos de treino.
2. O **comprimento médio** deve cair. As respostas de referência têm 96 palavras em média, bem menos do que o modelo base produz espontaneamente.
3. A **similaridade com a referência** deve subir modestamente. Uma subida grande seria indício de memorização, e não de generalização, com um dataset deste tamanho.
4. Os **trigramas distintos** podem cair no modelo treinado, porque fine-tuning em dataset pequeno aumenta a propensão a repetição.

Análise. Três das quatro hipóteses se confirmam na direção esperada. O comprimento médio cai de 189,8 para 107,1 palavras, a maior variação da tabela, e se aproxima das 96 palavras da resposta de referência. A similaridade com a referência sobe de 0,8993 para 0,9013. A subida é pequena, como previsto pelo piso alto do embedding descrito na seção 5.1. Os trigramas distintos caem de 0,9468 para 0,9389, uma queda leve, dentro do que a hipótese 4 previa como possível, não como certo.

A hipótese sobre a taxa de ressalva não se confirma. Ela cai de 0,2857 para 0,2143, na direção oposta à esperada. O comportamento mais diretamente ensinado pelo system prompt não aparece com mais frequência no modelo treinado. Uma hipótese é o corte de comprimento: a resposta fine-tuned é bem mais curta, e frases de ressalva podem estar entre o texto que deixou de ser gerado. Essa hipótese não foi verificada nos dados disponíveis.

A taxa de guardrail fica em 0,0 nos dois modelos. Nenhuma das 14 perguntas de validação aciona o filtro de prescrição, em nenhum dos dois lados, então essa métrica não separa base de fine-tuned neste conjunto de teste.

5.3 Guardrail de escopo

Medição sobre perguntas reais submetidas ao chat, com as similaridades registradas no log de auditoria:

Pergunta	Sim. clínica	Sim. fora	Margem	Decisão
"Devo trocar a vela de ignição do meu carro de 12 em 12 meses?"	0,847	0,921	-0,075	rejeitada
"Qual o melhor dia para comer feijoada?"	0,823	0,859	-0,036	rejeitada
"Caso eu queira colocar a sétima janela no meu carro, com quem devo falar?"	0,848	0,875	-0,027	rejeitada
"Qual cor de janela devo colocar em uma casa no campo?"	0,810	0,829	-0,019	rejeitada
"Devo lavar o olho com água nesse caso?"	0,848	0,841	+0,006	aceita
"Andar de bicicleta na Antártica tomando um sorvete é uma boa ideia?"	0,857	0,853	+0,005	aceita

Análise. As quatro perguntas claramente de outro domínio são rejeitadas, e a pergunta clínica legítima, sobre a conduta em um paciente com conjuntivite, é aceita. Os números absolutos confirmam o piso do modelo de embeddings: todas as similaridades contra as âncoras clínicas ficam entre 0,81 e 0,86, independentemente do assunto. É a comparação relativa que separa as classes.

A última linha expõe o limite da abordagem. Uma pergunta absurda, porém semanticamente neutra, é aceita com margem de +0,005. E a margem da pergunta clínica legítima é +0,006, da mesma ordem. **As duas classes não estão separadas por uma margem confortável.** Elevar o `SCOPE_MARGIN` para eliminar o falso positivo eliminaria também a pergunta clínica válida.

O guardrail resolve o caso comum, mas sem folga de decisão. As três similaridades ficam gravadas no log de auditoria, o que permite recalibrar o `SCOPE_MARGIN` com um volume maior de perguntas clínicas reais. Esse registro é, em si, um exemplo de explainability: a decisão de recusa é um número auditável, não uma opinião do modelo.

5.4 Qualidade da recuperação (RAG)

Distribuição das fontes citadas nas 34 recuperações registradas:

Protocolo	Vezes citado
<code>protocolo_seg04_contraste</code>	20
<code>protocolo_onco03_seguranca_quimioterapia</code>	5
<code>protocolo_mama02_conduta_birads</code>	5
<code>protocolo_mama01_rastreamento</code>	4
<code>protocolo_onco01_encaminhamento</code>	0

Um único protocolo concentra 57% das citações, inclusive em casos em que ele é irrelevante. À pergunta "Devo lavar o olho com água nesse caso?", para um paciente com conjuntivite, o sistema citou o protocolo de segurança no uso de contraste iodado.

Diagnóstico. O log revela o mecanismo por trás desse número: **29 das 32 interações com recuperação citaram exatamente uma fonte**, apesar de o retriever estar configurado com `top_k=4`. Um retriever que devolve 4 chunks e produz uma única fonte distinta está devolvendo o mesmo chunk quatro vezes.

A causa está na semântica do `Chroma.from_documents` sobre um `persist_directory` já existente: ele **acrescenta** os documentos à coleção em vez de substituí-la. Executar o `rag/ingest.py` repetidamente multiplica o índice. O índice medido continha 10 chunks por protocolo, cinco vezes os 2 chunks que cada documento produz com `chunk_size=500`. Com cinco cópias idênticas de cada trecho, os 4 vizinhos mais próximos de qualquer consulta tendem a ser as próprias cópias do trecho de maior similaridade, e a diversidade da recuperação colapsa para um documento por pergunta.

Isso explica as duas observações de uma vez. A concentração em um único protocolo e a citação de fonte irrelevante vêm do mesmo lugar: com apenas um documento efetivo por resposta, uma similaridade marginal vence sozinha, sem os outros três trechos para contrabalançar.

Consequência no texto da resposta. A recuperação irrelevante não fica contida no contexto. Ela molda a resposta. Uma pergunta sobre qual procedimento realizar num paciente com hérnia inguinal recebeu chunks de pneumonia e de oncologia mamária, e a resposta passou a percorrer o material recuperado comentando item por item, com frases como "*o protocolo X orienta...*", "*o protocolo Y não especifica...*" e "*os protocolos de pneumonia e mamário também não oferecem detalhes*", antes de

arriscar uma conduta ao final. O modelo tratou o contexto como uma lista a resenhar, e não como material para responder.

Dois defeitos se somaram aí. O primeiro é de recuperação, tratado pelo corte de relevância. O segundo é de **prompt**: o caminho do chat não dizia nada sobre a forma da resposta, ao contrário da visão geral, que sempre teve uma instrução de formato. Um modelo pequeno que recebe uma lista rotulada e nenhuma instrução tende a resenhar a lista. O `CHAT_INSTRUCTION`, descrito em 4.2, fecha essa lacuna.

Configuração atual. O `rag/ingest.py` reconstrói o índice do zero a cada execução, o que torna a indexação idempotente e elimina a duplicação. A base indexada é de 32 protocolos e 122 chunks, cobrindo as especialidades atendidas pelo hospital. A recuperação passou a aplicar o corte de relevância de `rag/relevance.py`, que devolve lista vazia quando nenhum candidato supera o piso.

Limitação que permanece. Os dois parâmetros do corte, a margem relativa e o piso absoluto, foram escolhidos por raciocínio sobre o comportamento do modelo de embeddings, e não por calibração sobre dado real. As similaridades passaram a ser gravadas no log de auditoria justamente para permitir esse ajuste depois. Um piso alto demais faz o assistente declarar falta de protocolo onde há um aplicável, e um piso baixo demais devolve o comportamento anterior.

Há um efeito colateral favorável em toda essa análise. Como as fontes são citadas, o problema ficou **visível** e mensurável a partir do log. Um sistema sem explainability teria o mesmo defeito, em silêncio.

5.5 Estabilidade da geração e aderência ao formato

A configuração de geração aplica `repetition_penalty=1.15` e `no_repeat_ngram_size=3` nos dois pipelines, tetos de tokens separados por tarefa (400 no chat e 450 na visão geral) e geração gulosa nos dois casos. Isso reduz, sem eliminar, dois comportamentos característicos de modelos pequenos: loops de repetição e truncamento no meio da resposta.

Vale registrar o limite dessas defesas. O `no_repeat_ngram_size` proíbe repetir a mesma sequência exata de três tokens, e por isso **não contém a deriva morfológica**. Quando o modelo produz `fibrose`, `fibrotic`, `fibrilari` e `fibrinoses`, cada item é uma sequência de tokens diferente, e nenhuma restrição de n-grama é violada. O teto de tokens é a contenção que efetivamente limita o volume dessa deriva, e por isso um teto generoso demais é contraproducente. Ele não dá espaço para uma resposta melhor, dá espaço para o modelo continuar gerando depois de já ter dito o que tinha a dizer.

A aderência ao formato de saída pedido à visão geral, com os marcadores `RELEVANTE:` e `ATENCAO:` e itens iniciados por `-`, não é garantida pelo modelo. O parser (`agent/overview_parser.py`) é defensivo por isso:

- **Rótulos tolerantes a variação.** Aceita singular e plural, com e sem acento, pelos padrões `RELEVANTES?` e `ATEN[CÇ][AÃ]0(?:ES|S)?`.
- **Três níveis de fallback na extração de itens.** Primeiro os marcadores `-` e `*`, depois uma linha por item quando não há marcador, e por fim a enumeração em linha única separada por vírgula ou ponto-e-vírgula.
- **Consolidação de itens redundantes** (`agent/overview_summarizer.py`). Quando vários itens compartilham prefixo e sufixo em nível de palavra, eles são fundidos em um só, com as variações

separadas por vírgula. A fusão é conservadora e nunca junta achados sem sobreposição textual clara.

O princípio de projeto é tratar o formato como saída não confiável e parseá-la defensivamente, em vez de depender da obediência do modelo ao prompt. Cada regra do parser tem teste de regressão correspondente.

A limitação estrutural do parser é que ele trata **forma**, e não **conteúdo**. Uma lista de 167 itens bem formatados, com marcadores corretos, atravessa todas as suas regras sem objeção. Essa é a fronteira tratada na seção seguinte.

5.6 Fidelidade ao contexto

A limitação mais séria do modelo aparece na fidelidade ao caso do paciente, e ela se manifestou de quatro formas distintas.

Comorbidade fabricada. Trecho do bloco de pontos de atenção gerado para um paciente com conjuntivite bacteriana:

"Hipertensão arterial sistêmica alta (PA 180/100 mmHg). Não há histórico de diabetes ou colesterol alto. **Há suspeita de reação adversa a antibióticos. Pacientemente é portador de doença autoimune. Paciência tem histórico de problemas cardíacos familiares.** Paciente TEM HIPERTENSÃO ARTERIAL SISTÊMICA ALTA E CONSIDERA APOIAR SEUS DADOS CLÍNICOS COM UMA TECNOLOGIA DE COORDENAÇÃO DOS DADOS DO PACIENTE..."

Doença autoimune, histórico familiar cardíaco e suspeita de reação adversa a antibióticos não constam da ficha. Há ainda degeneração morfológica, com "Paciente" virando "Pacientemente" e depois "Paciência", e "mmHg" virando "mmHgg". A frase final perde o sentido e sai em caixa alta.

Deriva por associação livre. A manifestação extrema apareceu num paciente com hérnia inguinal não complicada, cujo bloco de pontos de atenção veio com **167 itens**. A sequência começa em um achado plausível, "Encarceramento", que é a complicação real de uma hérnia. Depois passa por comorbidades não registradas na ficha, como insuficiência renal crônica, diabetes e edema pulmonar. Em seguida deriva para famílias inteiras de neoplasias e infecções sem qualquer relação com o caso, como **carcinoma epitelial, carcinoma laringe, meningovascular e meningococemia**. E termina em morfologia inventada: **fibrilari, filtração, filtrança, filtrapta, filtreiras e filtramentos**.

O padrão tem três marcas mensuráveis. O **volume** fica muito acima dos 5 itens pedidos. Há **deriva temática**, já que nenhum dos 167 itens compartilha um termo de conteúdo com o prontuário. E aparecem **famílias morfológicas**, que são sequências de variações de um mesmo radical.

Conduta na faixa etária errada. Esta manifestação tem causa distinta das duas anteriores. À pergunta sobre qual procedimento realizar num paciente de **66 anos** com hérnia inguinal, a resposta descreveu a conduta da hérnia inguinal **pediátrica**.

Aqui o modelo não fabricou, ele preencheu uma lacuna. Idade e sexo não constavam do contexto enviado ao prompt. A função `montar_contexto_clinico` compunha o bloco a partir do histórico resumido e dos campos da anamnese, e nenhum dos dois carrega esses dados, que vivem na tabela

pacientes . Sem a idade, o modelo assumiu a faixa etária mais frequente na literatura do diagnóstico, e a hérnia inguinal pediátrica é largamente predominante nessa literatura. O resultado é o tipo de erro mais difícil de perceber numa leitura rápida: internamente coerente, clinicamente bem escrito e errado para o paciente à frente.

A correção é na origem, com o contexto passando a abrir com idade e sexo, conforme descrito em 3.3. A checagem de coerência etária de 3.6 é a segunda camada, porque a consequência de não perceber esse erro é alta demais para depender de uma única defesa. Aplicada à resposta observada, com a idade real do paciente:

Entrada	Resultado
Resposta citando <code>crianças</code> , paciente de 66 anos	<code>requer_validacao_humana=true</code> , termo <code>crianças</code> reportado, ressalva anexada
Mesma resposta, paciente de 4 anos	não sinalizada, porque a faixa citada contém a idade
"diferente do que ocorre em crianças, no idoso..." , paciente de 66 anos	não sinalizada, porque há faixa compatível na resposta

Citação falsa. A quarta manifestação está na mesma resposta sobre a hérnia inguinal:

"Protocolo seg-01: Hernias inguinais, orienta-se a realização de uma cirurgia [...] **Protocolo pneumo-02:** as recomendações são generalizadas [...] parece conveniente realizar uma **cirurgia eletiva emergencial** [...] justificado pelos **sinais potenciais de estrangulamento**"

São quatro erros distintos, de naturezas diferentes:

Trecho	Problema
"Protocolo seg-01: Hernias inguinais"	O SEG-01 existe, mas trata de alergias e interações medicamentosas. Hérnia inguinal é o CIR-01
"Protocolo pneumo-02"	Código inexistente. Os de pneumologia são PNE-01 a PNE-03
"cirurgia eletiva emergencial"	Contradição em termos, e a resposta antes sugerira monitoramento domiciliar
"sinais potenciais de estrangulamento"	A ficha diz o oposto: "sem sinais de encarceramento ou irritação peritoneal"

A verificação de citações de 3.6 cobre os dois primeiros. Aplicada a essa resposta, ela sinaliza `SEG-01` e `PNEUM0-02` como citações que não estão nas fontes consultadas, e marca a resposta para validação humana. A contradição interna e a contradição com a ficha são tratadas pelo prompt, nas regras 4 e 5 do `CHAT_INSTRUCTION` , e não por verificação automática. Detectá-las exigiria comparação semântica entre afirmações, o que está fora do alcance das camadas léxicas deste projeto.

Por que as camadas anteriores não contêm a deriva. O guardrail de prescrição não sinaliza esse texto, e corretamente, porque não há linguagem de prescrição nele. Ele cobre *prescrição direta*, e não *fabricação de fato clínico*, que são riscos diferentes. O `no_repeat_ngram_size` não se aplica, porque cada item é uma sequência de tokens distinta. O `merge_similar_bullets` também não, porque exige prefixo e sufixo comuns em nível de palavra, e itens de uma ou duas palavras não formam template. E o parser não objeta, porque a lista está bem formatada.

A camada que trata disso. O filtro de ancoragem (`agent/overview_filter.py` , descrito em 3.6) exige que cada item compartilhe um termo de conteúdo com o prontuário do paciente, e descarta o bloco inteiro quando classifica a geração como degenerada. Aplicado à saída de 167 itens acima, com o contexto clínico real do paciente:

Bloco	Itens gerados	Ancorados no prontuário	Resultado
RELEVANTE	2	2	ambos exibidos
ATENCAO	167	0	bloco descartado, <code>geracao_degenerada=true</code>

O médico vê os dois pontos relevantes legítimos e nenhum ponto de atenção, em vez de uma lista de 167 termos entre os quais não teria como distinguir o achado real do inventado. A API devolve `pontos_descartados` e `geracao_degenerada` para que a interface possa diferenciar "nada a sinalizar" de "resposta descartada".

O que permanece. As quatro manifestações têm a mesma origem, um modelo pequeno ao qual se pede síntese e raciocínio clínico, mas exigiram defesas distintas. Dado ausente do prompt se resolve completando o prompt. Conteúdo fabricado se resolve exigindo ancoragem. Conduta na faixa errada se resolve confrontando a resposta com o dado estruturado. Citação falsa se resolve conferindo a referência contra as fontes efetivamente consultadas.

Nenhuma dessas defesas ataca a geração. O filtro impede a exibição da alucinação, não impede a produção dela. A causa é estrutural: um modelo de 1,5 bilhão de parâmetros ajustado com poucas dezenas de exemplos, ao qual se pede síntese analítica sobre um bloco de texto longo. O fine-tuning ensina o formato, não a fidelidade ao contexto. Um item de atenção clinicamente correto, mas expresso com vocabulário ausente da ficha, também é descartado. É uma perda real e assumida, porque a alternativa é exibir uma lista cuja confiabilidade o leitor não pode avaliar.

5.7 Comportamento agregado do sistema

Das 68 interações registradas, sobre 7 pacientes distintos:

Indicador	Valor	Leitura
Interações totais	68	31 visões gerais, 24 chats e 13 anteriores ao registro de tipo
Sinalizadas pelo guardrail de prescrição	0	Nenhum falso positivo, e também nenhum verdadeiro positivo
Coerência etária	Camada nova	Sem histórico de operação. Validada por teste sobre o caso real
Verificação de citações	Camada nova	Também sem histórico. Sinaliza as duas citações falsas do caso observado
Encaminhadas para validação humana	0	Consequência direta da linha do guardrail de prescrição
Com alerta de exame pendente	34 (50%)	O nó de exames está exercitado e funcionando
Erros de execução do grafo	0	Todas chegaram a <code>status="concluido"</code>

Sobre o guardrail de prescrição nunca ter disparado. Isso não é evidência de que ele funciona. É evidência de que o modelo treinado não produziu linguagem de prescrição direta nas perguntas submetidas. Esse é o comportamento desejado, mas também significa que o filtro **não foi exercitado em operação**. Sua cobertura real é conhecida apenas pelos testes unitários, e o caminho de validação humana no grafo permanece não exercitado com dado real.

Cobertura de testes. São 86 testes automatizados, todos passando, sobre lógica pura, sem carregar LLM nem modelo de embeddings. A distribuição é: guardrail de escopo com 7, consolidação de itens com 7, parsing da visão geral com 11, filtro de ancoragem com 15, coerência etária com 15, verificação de citações e corte de relevância com 18, e métricas de avaliação com 12. Cada caso real observado em operação entra como teste de regressão: a saída degenerada de 167 itens, a resposta pediátrica para paciente de 66 anos e as citações `SEG-01` e `PNEUM0-02`.

5.8 Síntese e limitações

Aspecto	Estado	Evidência
Pipeline ponta a ponta	✓ Funcional	68 interações, 0 erros de execução
Fine-tuning	✓ Pipeline completo e reproduzível	Dataset de 98 exemplos, QLoRA em 33 passos, adapter de 17,4 MB
Explainability	✓ Implementada	Fontes citadas em toda resposta de chat e similaridades numéricas no log
Prontuário multiespecialidade	✓ Implementado	32 pacientes, 30 quadros clínicos distintos
Cobertura da base de conhecimento	✓ Alinhada ao escopo	32 protocolos e 43 FAQs internas cobrindo as áreas do prontuário
Indexação do RAG	✓ Idempotente	O <code>reset_vectorstore</code> elimina a duplicação de chunks
Citação de protocolo	✓ Verificada	Código inexistente ou não recuperado é sinalizado
Forma da resposta do chat	✓ Especificada	O <code>CHAT_INSTRUCTION</code> define conduta direta, citação e sinais de alerta
Exibição de conteúdo fabricado	✓ Bloqueada	O filtro de ancoragem descarta o bloco degenerado, de 167 para 0 itens
Dado demográfico no prompt	✓ Corrigido	Idade e sexo abrem o contexto clínico
Guardrail de escopo	⚠ Funciona, sem folga	Margens de +0,005 no falso positivo contra +0,006 no caso legítimo
Corte de relevância	⚠ Aplicado, não calibrado	Margem e piso escolhidos por raciocínio, sem medição
Estabilidade da geração	⚠ Contida por parsing defensivo	Repetição e desvio de formato tratados no backend
Guardrail de prescrição	⚠ Não exercitado	0 acionamentos em 68 interações
Conduta em faixa etária errada	⚠ Sinalizada, não bloqueada	A checagem de coerência etária marca para validação humana

Aspecto	Estado	Evidência
Geração de conteúdo fabricado	✗ Limitação do modelo	A alucinação ocorre. É contida na exibição, não na origem
Comparação base vs. fine-tuned	✓ Concluída	Ressalva cai, na direção oposta à hipótese. Comprimento, similaridade e trigramas confirmam a direção prevista, conforme 5.2

Limitações:

1. **Fidelidade ao contexto na visão geral**, descrita em 5.6, é a mais grave. O filtro de ancoragem impede que conteúdo fabricado seja exibido, mas não impede que seja gerado, e descarta junto a inferência clínica legítima que use vocabulário ausente da ficha.
2. **Dataset desalinhado com uma das tarefas**. Os 98 exemplos são de resposta a pergunta clínica e de geração de documento. Nenhum é de síntese de prontuário, que é a tarefa da visão geral.
3. **Volume do dataset**. 98 exemplos são suficientes para ajustar formato e tom, não para incorporar conhecimento clínico novo. Isso é coerente com a divisão de papéis da arquitetura, mas limita o que se pode esperar do fine-tuning isoladamente.
4. **Tradução automática sem revisão clínica** nas 50 FAQs derivadas de bases públicas.
5. **Protocolos sintéticos**. Os 32 protocolos são material acadêmico escrito para o projeto, não diretrizes institucionais validadas.
6. **Coerência etária apenas lexical**. A checagem cobre termos explícitos de faixa etária. Uma conduta inadequada à idade expressa sem nenhum desses termos passa sem sinalização.
7. **Verificação de citações apenas referencial**. Ela confere se o protocolo citado foi consultado, não se a resposta descreve corretamente o que ele diz.
8. **Contradição interna não é detectada**. Uma resposta que sustenta duas condutas incompatíveis, ou que contradiz a ficha do paciente, é tratada só pelo prompt.
9. **Corte de relevância não calibrado**. Margem e piso foram escolhidos por raciocínio sobre o comportamento do E5, e não por medição, conforme 5.4.
10. **Guardrail de escopo sem folga de decisão**, conforme 5.3.
11. **Guardrail de prescrição não exercitado em operação**, conforme 5.7.
12. **Avaliação sem julgamento clínico**. Nenhuma métrica deste trabalho mede correção médica.
13. **Sem persistência de estado entre interações**. Não há memória de conversa.
14. **Modelo servido em processo**. O `HuggingFacePipeline` roda dentro do FastAPI, o que é adequado ao escopo acadêmico, mas não a produção.

Ações indicadas, por prioridade:

#	Ação	Justificativa
1	Calibrar margem e piso do corte de relevância com as similaridades do log	Os valores atuais não foram medidos, conforme 5.4
2	Ampliar o dataset com exemplos de síntese de prontuário	A tarefa da visão geral não tem representação no treino, o que é a raiz da deriva descrita em 5.6
3	Calibrar o filtro de ancoragem com dado de operação	Medir quantos itens legítimos estão sendo descartados junto com os fabricados
4	Recalibrar o <code>SCOPE_MARGIN</code> com volume de perguntas clínicas reais	Requer dado de operação ainda não disponível

#	Ação	Justificativa
5	Testar Qwen2.5-3B ou 7B	Ataca a alucinação na origem, e não apenas na exibição
6	Servir o modelo fora do processo, com merge e GGUF no Ollama, ou com vLLM	Desacopla API e inferência

5.9 Estado de medição

O dataset e a base de protocolos foram ampliados para cobrir todas as especialidades do hospital. Isso torna necessária a regeneração dos dois artefatos derivados antes que as métricas sejam refeitas:

```
python rag/ingest.py          # reindexa os 32 protocolos em 122 chunks
python finetuning/train_qlora.py # re-treina com os 84 exemplos
python finetuning/evaluate.py  # preenche a tabela de 5.2
```

Medição	Situação
Comparação base vs. fine-tuned (5.2)	Concluída, sobre o adapter re-treinado com os 84 exemplos
Guardrail de escopo (5.3)	Válida. O <code>agent/scope_guard.py</code> , as âncoras e o modelo de embeddings não mudaram
Distribuição de fontes (5.4)	A refazer depois da reindexação. A análise de causa permanece válida. Medir também quantas perguntas passam a cair no caso "sem protocolo aplicável"
Estabilidade e parsing (5.5)	A refazer, porque o teto de tokens e a decodificação foram alterados
Fidelidade ao contexto (5.6)	O comportamento do filtro foi verificado sobre a saída real. A taxa de descarte de itens legítimos ainda não foi medida
Coerência etária (5.6)	Sem dado de operação. Medir a taxa de acionamento e de falso positivo
Verificação de citações (5.6)	Também sem dado de operação. Medir quantas respostas citam protocolo não consultado
Comportamento agregado (5.7)	A refazer com o novo acervo em operação

6. Referências e licenças

Dados

- **MedQuAD.** Ben Abacha, A. e Demner-Fushman, D. "A Question-Entailment Approach to Question Answering." *BMC Bioinformatics*, 2019. Licença **CC BY 4.0**. <https://github.com/abachaa/MedQuAD>
- **PubMedQA.** Jin, Q. et al. "PubMedQA: A Dataset for Biomedical Research Question Answering." 2019. Licença **MIT**. <https://github.com/pubmedqa/pubmedqa>

Conteúdo derivado, filtrado e vertido para o português com apoio de LLM. A atribuição também está registrada em [1.AssistenteMedico/data/raw/faqs/_FONTE.md](#).

Modelos

- **Qwen2.5-1.5B-Instruct.** Qwen Team, Alibaba Cloud. Licença Apache 2.0.
<https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct>
- **multilingual-e5-small.** Wang, L. et al. "Multilingual E5 Text Embeddings." Licença MIT.
<https://huggingface.co/intfloat/multilingual-e5-small>

Técnicas

- **LoRA.** Hu, E. et al. "LoRA: Low-Rank Adaptation of Large Language Models." ICLR 2022.
- **QLoRA.** Dettmers, T. et al. "QLoRA: Efficient Finetuning of Quantized LLMs." NeurIPS 2023.

Bibliotecas

`transformers`, `peft 0.20`, `trl 1.12`, `bitsandbytes`, `accelerate`, `datasets`, `langchain`, `langchain-chroma`, `langchain-huggingface`, `langgraph`, `chromadb`, `sentence-transformers`, `fastapi`, `pydantic`, `uvicorn`, `pytest`, Angular 20 e Angular Material 20.

Documentos do projeto

- [README.md](#) : visão geral do repositório
- [1.AssistenteMedico/README.md](#) : backend, com execução, endpoints e avaliação
- [front/README.md](#) : frontend Angular
- [1.AssistenteMedico/data/raw/README.md](#) : origem e licenciamento dos dados

Aviso

Este é um projeto acadêmico de pós-graduação. O assistente é uma ferramenta de **apoio** à decisão clínica, e nenhuma resposta deve ser usada como prescrição sem validação de um profissional de saúde habilitado. Os dados de treino foram vertidos para o português automaticamente, sem revisão clínica. O sistema apresenta alucinação documentada na seção 5.6 e **não é adequado para uso clínico real**.